

Tutorial 1:

NumPy Fundamentals

Administrative: Assignment 1

Due 1/23 11:59pm

- K-Nearest Neighbor
- Linear classifiers: SVM, Softmax

Administrative: EdStem

Please make sure to check and read all pinned EdStem posts.

Batched Matrix Multiplication (matmul)

Matmul Operator: @

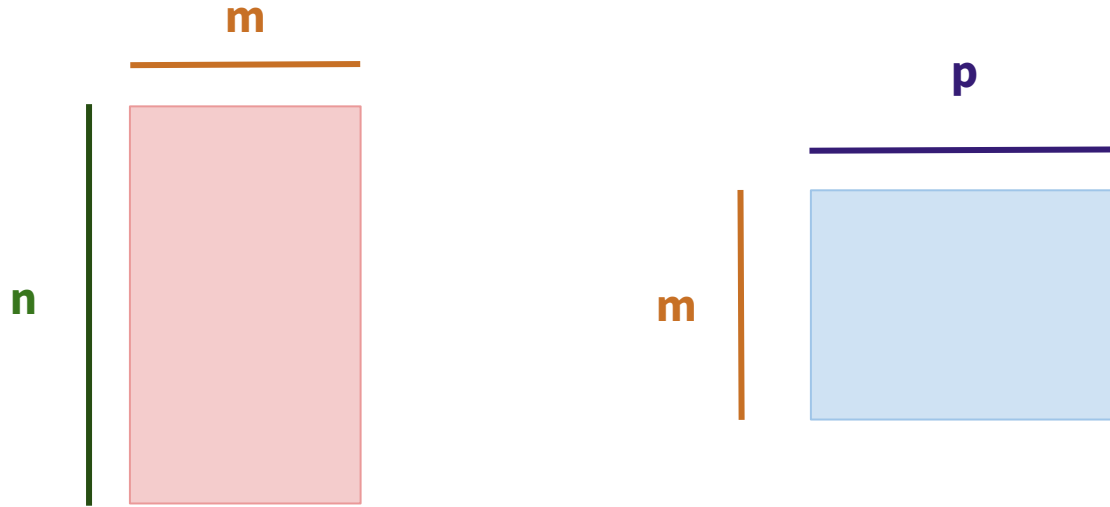
Assume **a** and **b** have been defined as NumPy matrices.

To perform a matrix multiplication and store the result in a new NumPy array **c**:

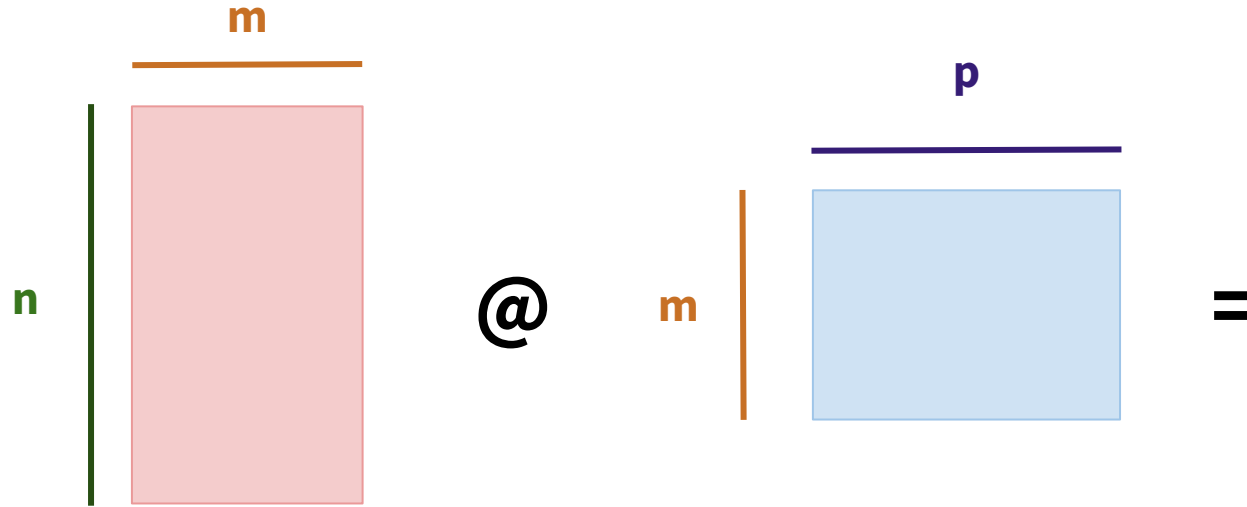
```
c = np.matmul(a, b)
```

```
c = a @ b
```

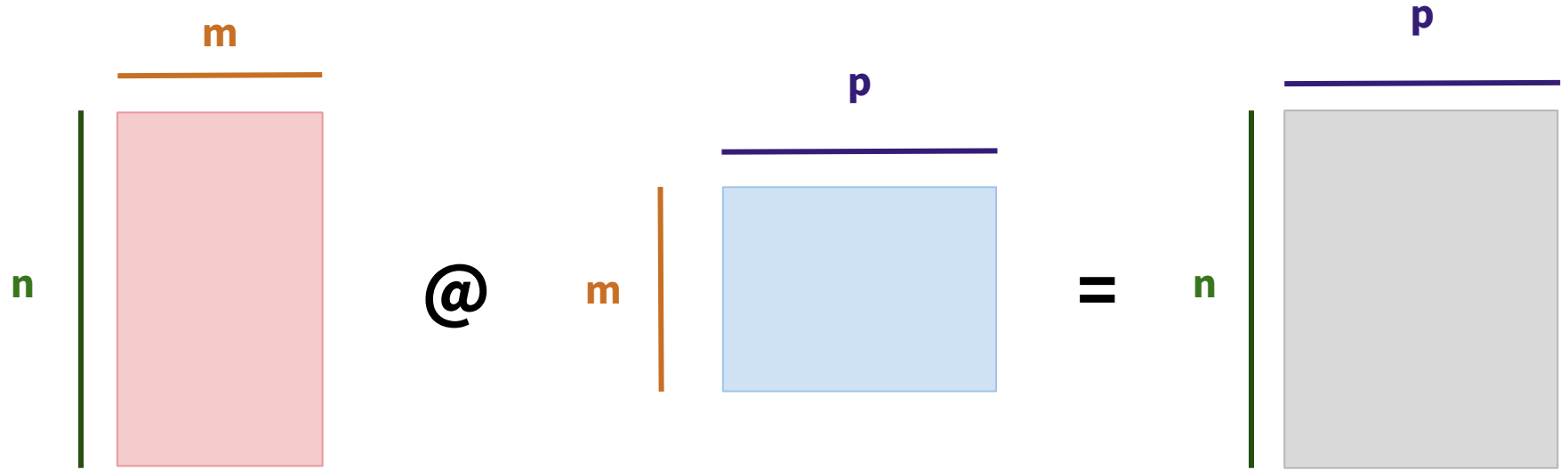
Matrix Multiplication



Matrix Multiplication

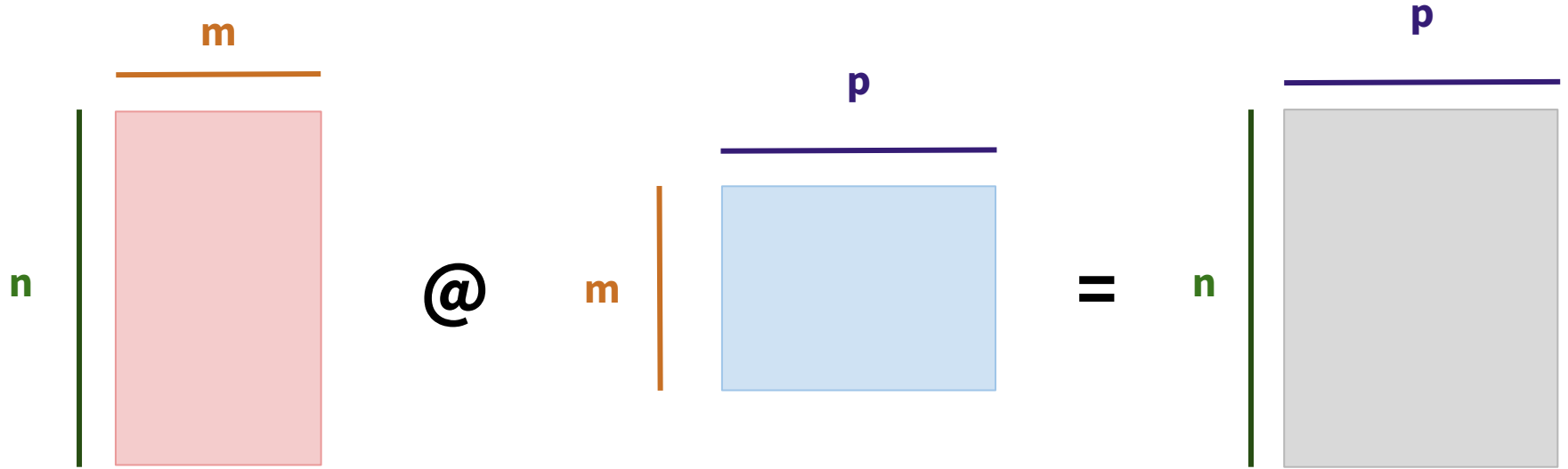


Matrix Multiplication



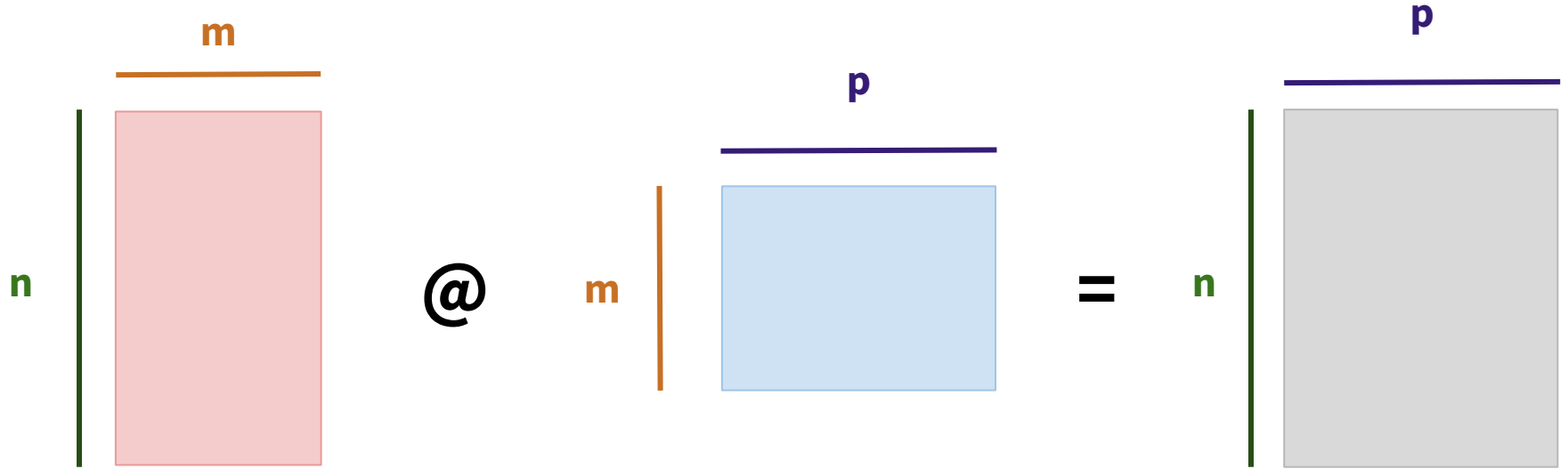
Matrix Multiplication

$$(n \times m) @ (m \times p) = (n \times p)$$



Matrix Multiplication

$$(n, m) @ (m, p) = (n, p)$$



Matrix Multiplication

$$(n, m) @ (m, p) = (n, p)$$

Batched Matrix Multiplication

$$\text{\#1} \quad (n, m) @ (m, p) = (n, p)$$

$$\text{\#2} \quad (n, m) @ (m, p) = (n, p)$$

⋮

...

$$\text{\#b} \quad (n, m) @ (m, p) = (n, p)$$

Batched Matrix Multiplication

$$(b, n, m) @ (b, m, p) = (b, n, p)$$

Batched Matrix Multiplication

$$(b_1, b_2, n, m) @ (b_1, b_2, m, p) = (b_1, b_2, n, p)$$

Batched Matrix Multiplication

Last 2 dimensions follow good ol'
matrix multiplication rules

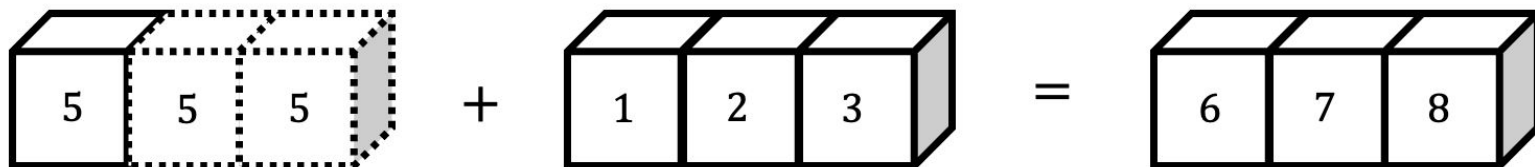
$$(b_1, b_2, n, m) @ (b_1, b_2, m, p) = (b_1, b_2, n, p)$$

All preceding dimensions follow
broadcasting rules

Broadcasting

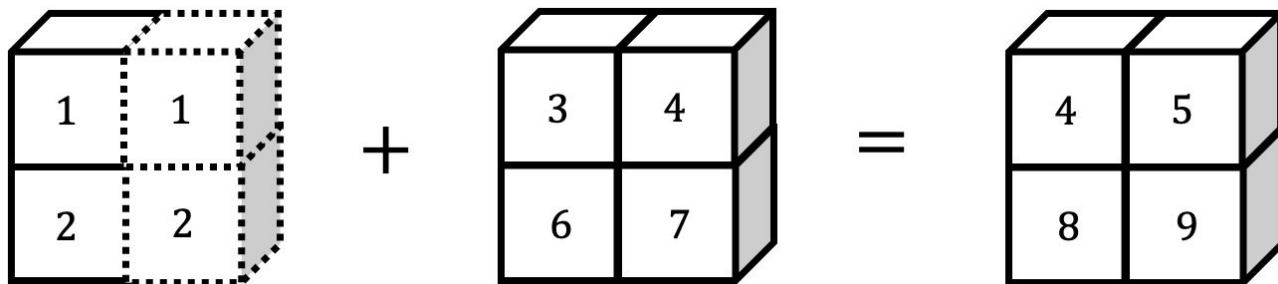
Broadcasting

```
5 + np.array([1, 2, 3])  
✓ 0.0s  
array([6, 7, 8])
```



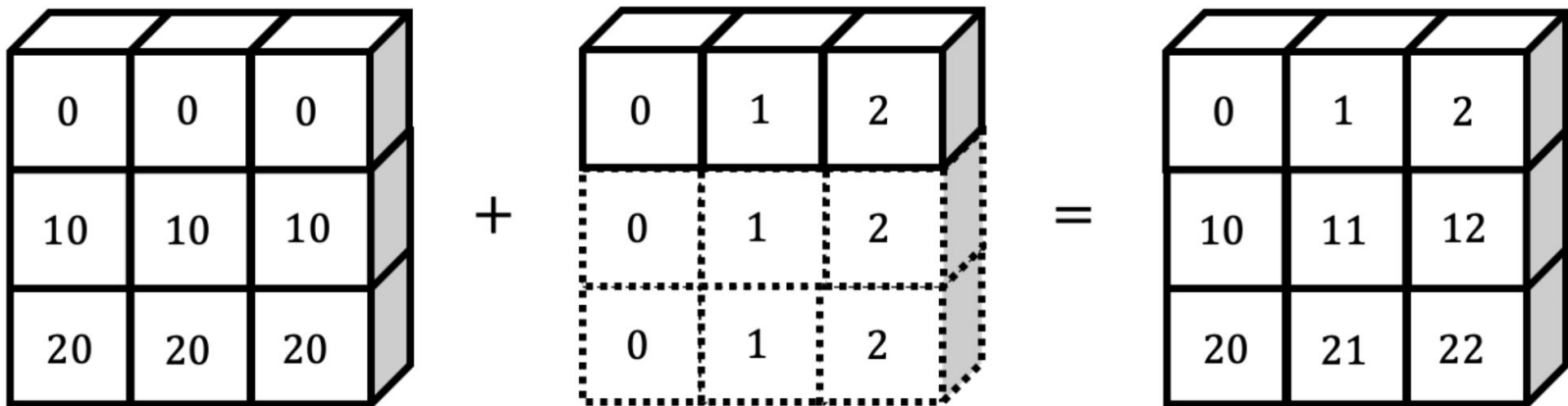
Broadcasting

```
np.array([[1], [2]]) + np.array([[3], [7]])  
✓ 0.0s  
array([[4, 5],  
       [8, 9]])
```



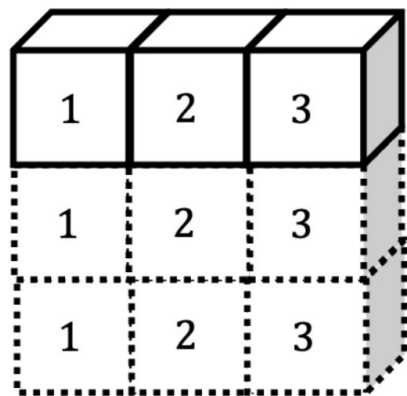
Broadcasting

```
np.array([[0, 0, 0], [10, 10, 10], [20, 20, 20]]) + np.array([0, 1, 2])  
✓ 0.0s  
array([[ 0,  1,  2],  
       [10, 11, 12],  
       [20, 21, 22]])
```



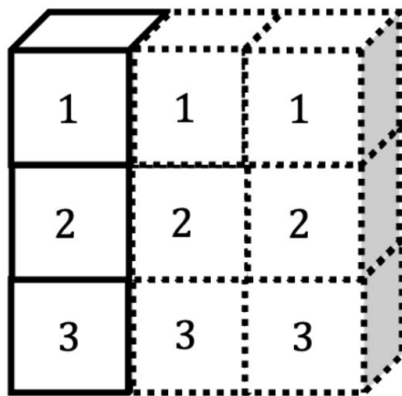
Broadcasting

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$



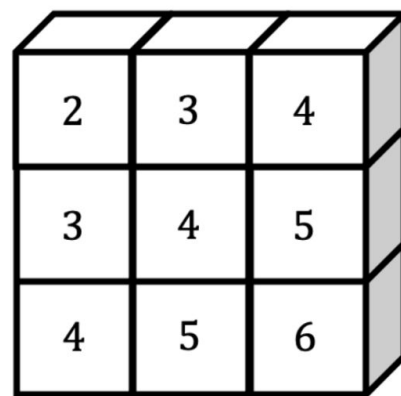
1	2	3
1	2	3
1	2	3

+



1	1	1
2	2	2
3	3	3

=



2	3	4
3	4	5
4	5	6

Broadcasting

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = \text{ERROR}$$

```
np.array([1, 2, 3]) + np.array([1, 2], [3, 4], [5, 6])
```

⊗ 0.0s

```
ValueError                                Traceback (most recent call last)
Cell In[27], line 1
----> 1 np.array([1, 2, 3]) + np.array([1, 2], [3, 4], [5, 6])

ValueError: operands could not be broadcast together with shapes (1,3) (3,2)
```

Broadcasting

1	2	3
1	2	3
1	2	3

+

3	4
4	5
5	6

You might be tempted to insert a dimension here, but which column would you copy?

=

INCOMPATIBLE

WHY?

Broadcasting

- What are the rules?
- Why use it?

Broadcasting

1. If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side.
2. If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape.
3. If in any dimension the sizes disagree and neither is equal to 1, an error is raised.

From Jake VanderPlas' [*Python Data Science Handbook*](#)

Broadcasting

You are given two arrays with different dimensions (“dims”, “axes”).

1. Pad the array with fewer **number** of dims with 1s on LHS until # of dims match.
2. Loop through each axis. If **exactly** one of the two arrays has size 1 in that axis, stretch it to match the other array.
3. If the array's shapes aren't exactly the same by now, throw an error.

Broadcasting

Note: rest of this section follows the worksheet that was handed out during class